DONANIM + YAZILIM REFERANS BELGESI

LinoRobot2 Yapım Kılavuzu

ESP32-S3 ve Raspberry Pi 4 üzerine, `linorobot2` / `linorobot2_hardware` açık kaynak ekosistemiyle inşa edilmiş, özel (custom) mekanik parçalara sahip bir diferansiyel sürüşlü mobil robotun sıfırdan yeniden üretilebilmesi için gereken malzeme listesi, güç mimarisi, kablolama şeması, yazılım kurulumu ve kalibrasyon adımlarının tamamı.

TABAN	MCU	BILGISAYAR	LIDAR	BATARYA	YAZILIM
2WD / Diferansiyel	ESP32-S3-DevKitC-1	Raspberry Pi 4	RPLIDAR A1M8 / S1	3S1P 11.1V LiPo	ROS 2 Humble

§01 Genel Bakış

Bu robot, TurtleBot3 Burger sınıfına yakın ölçülerde ama tamamen özel tasarlanmış bir platformdur: mekanik parçalar (motor oturağı, gövde katmanları, tekerlek ve şaft adaptörü) 3D yazıcıdan basılır; elektronik, firmware ve yazılım tarafı bu kılavuzda anlatılan açık kaynak yığınla kurulur.

Sistem mimarisi — üç katman

Kontrol üç ayrı katmana bölünmüştür, her biri farklı bir gerçek-zamanlılık ihtiyacını karşılar:

KATMAN	GÖREV	NEDEN AYRI
ESP32-S3 (firmware)	20ms/50Hz döngüyle motor PID, encoder okuma, IMU/odometri yayını	Gerçek zamanlı, kesintisiz döngü gerektirir — bir işletim sistemi üzerinde garanti edilemez
Raspberry Pi 4 (robot bilgisayarı)	micro-ROS agent, LIDAR sürücüsü, EKF, (opsiyonel) SLAM/Nav2	ROS 2 çalıştırabilecek güçte ama robot üzerinde taşınabilir, düşük güçlü
PC / Workstation (opsiyonel)	RViz görselleştirme, ağır Nav2/SLAM hesaplaması	RPi4'ün CPU sınırını aşan görevler için — bkz. §10

> TASARIM FELSEFESI
>
> Her teknik karar mümkün olduğunca resmi datasheet veya kaynak koda dayandırılmalı, tahmine dayanmamalıdır. Bu kılavuzdaki her voltaj/akım değeri ilgili üreticinin (Pololu, Slamtec, Infineon/BTS7960 modül üreticisi, Raspberry Pi Vakfı) resmi belgelerinden alınmıştır.

§02 Malzeme Listesi

Elektronik ve hesaplama bileşenleri hazır/satın alınan parçalardır; mekanik gövde ise özel tasarımdır (bu kılavuzda genel ilkeleri anlatılır, CAD dosyası kapsam dışıdır).

HESAPLAMA & KONTROL

BILEŞEN	MODEL	ADET	NOT
Mikrodenetleyici	ESP32-S3-DevKitC-1 (WROOM-1, N8R2)	1	8MB flash / 2MB PSRAM, dahili native USB — harici USB-TTL adaptörü gerekmez
Robot bilgisayarı	Raspberry Pi 4 Model B, 4GB	1	Ubuntu Server 22.04.5 LTS 64-bit (Desktop değil — RAM tasarrufu)
microSD kart	≥32GB, Class 10 / A2	1	—

TAHRIK (MOTOR & SÜRÜCÜ)

BILEŞEN	MODEL	ADET	NOT
Redüktörlü DC motor	Pololu 25D, 99:1, #3219	2	12V, 100 RPM (yüksüz), dahili 48 CPR Hall encoder, 4mm D-şaft
Motor sürücüsü	BTS7960 (IBT_2 modül kartı)	2	Motor başına 1 sürücü — H-köprü, PWM kontrollü, 6-27V aralığı
Tekerlek + şaft adaptörü	Özel tasarım (3D baskı)	2 + 2	Bkz. §05 — şaft oturma toleransı kritik

ALGILAMA

BILEŞEN	MODEL	ADET	NOT
2D LIDAR	RPLIDAR A1M8 (veya S1)	1	Adaptör kartlı (development kit) versiyon — tek USB kablosu yeterli

GÜÇ

BILEŞEN	MODEL	ADET	NOT
Batarya	3S1P LiPo, 11.1V, ≥1800mAh, ≥5C	1	C değeri ne kadar yüksekse o kadar iyi — bkz. §03 risk notu
Step-down (buck) regülatör	XL4015E1 modül, ~5A	1	12V(11.1V) → 5V, tüm lojik/bilgisayar hattının tek kaynağı
Bağlantı	Batarya konnektörü, USB Type-C kablo (kesilecek)	1+1	Type-C'den sadece Kırmızı(+5V)/Siyah(GND) lehimlenir

MEKANIK & SARF

BILEŞEN	NOT
Gövde katmanları, motor oturağı, sürücü braketleri	3D baskı, özel tasarım — bkz. §05
Distans (spacer) çubuklar + M3 somun	Katmanlar arası montaj için
M3 vida/somun seti, 1.5mm Allen anahtar	Genel montaj + şaft sabitleme
Multimetre	Alet — voltaj/akım doğrulaması için zorunlu, bkz. §09

§03 Güç Mimarisi

Tek bir batarya, tek bir regülatör (XL4015E1) üzerinden tüm sistemi besler. Motorlar ham (regüle edilmemiş) batarya gerilimini alır; RPi4 ve sürücü lojik hatları regüle 5V'u paylaşır; encoderlar ESP32-S3'ün kendi 3.3V pininden beslenir (5V'tan **asla** değil — GPIO'lar 3.3V'a dayanıklıdır).

Güç bütçesi (üretici datasheet'lerinden)

BILEŞEN	GERILIM	TIPIK AKIM	AZAMI / AÇILIŞ AKIMI
Raspberry Pi 4	5V (USB-C)	0.21–0.65A (ölçülen, boşta–tam yük)	3A (resmi giriş özelliği)
ESP32-S3-DevKitC-1	5V (RPi4 USB üzerinden)	~0.25A	–
RPLIDAR A1M8 (tarayıcı)	5V	300–350mA	500–600mA (açılış)
RPLIDAR A1M8 (motor)	5V (adaptör kartı üzerinden dahili)	~100mA	–
Pololu 25D motor (×2)	11.1V (ham)	~150–300mA / motor (normal sürüş)	~5.6A / motor (kilitlenme/stall)
BTS7960 sürücü (×2)	5V lojik + 11.1V güç	~50mA lojik / sürücü	Modül reyting: ~43A/kanal (kısa süreli, kullanılmıyor)
Toplam (bataryaya yansıyan, en kötü durum)	**11.1V eşdeğer**	**~12.6A (iki motor aynı anda kilitlenirse)**	

BILINEN RISK

5C'lik bir bataryanın (ör. 1800mAh × 5C = 9A azami) sürekli akım kapasitesi, iki motorun aynı anda kilitlenmesi (engele çarpma) senaryosundaki ~11.2A ihtiyacın **altında** kalır. Firmware'de akım sınırlama yoktur (`BATTERY_PIN` / `USE_INA219` tanımlı ama kullanılmaz — bkz. §10). Mümkünse ≥10C, tercihen ≥20C bir batarya seçilmesi önerilir; düşük C'li bir batarya kullanılacaksa risk bilinçli olarak kabul edilmelidir.

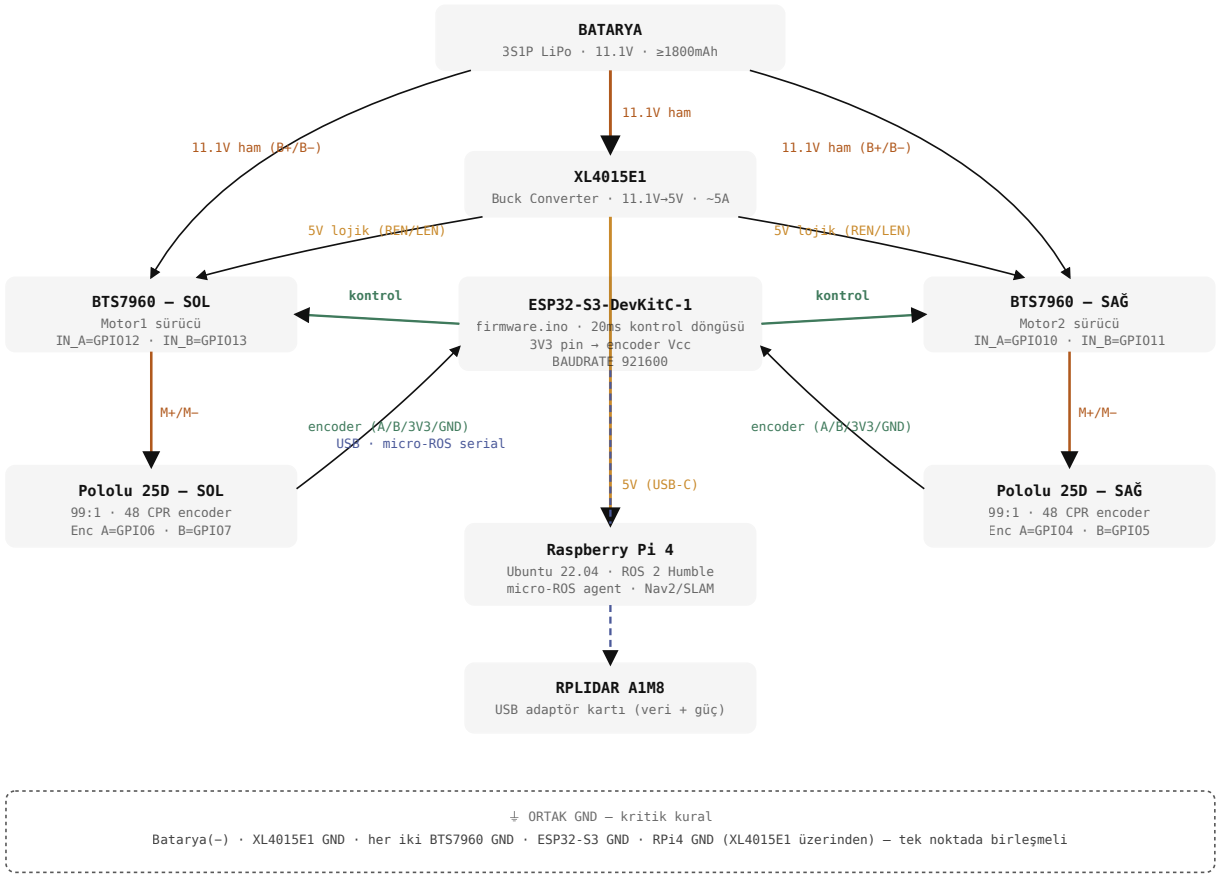
Güç dağıtım noktası — XL4015E1

XL4015E1'in giriş ve çıkış tarafı her biri ikiye ayrılır:

TARAF	KAYNAK/HEDEF	GERILIM
Giriş	Batarya (+/−)	11.1V ham
	BTS7960 ×2, B+/B− (motor güç girişi)	11.1V ham
Çıkış (regüle)	Raspberry Pi 4, USB-C güç girişi	5V
	BTS7960 ×2, REN/LEN + VCC (lojik/etkinleştirme)	5V

§04 Kablolama Şeması

Aşağıdaki şema, güç hatlarını (renk kodlu) ve sinyal/veri bağlantılarını tek diyagramda gösterir. Tüm pin numaraları `esp32s3_config.h` kaynağından doğrudan alınmıştır.



Güç, batarya(11.1V)'dan hem doğrudan iki BTS7960 sürücüsüne hem de XL4015E1 üzerinden 5V'a düşürülüp RPi4 ve sürücü lojik hatlarına dağılır. ESP32-S3, her iki sürücüyü kontrol sinyaliyle yönetir ve her iki motorun encoderini okur; RPi4 ile arasında ayrı bir USB veri hattı (micro-ROS) vardır.

Tam pin haritası

İŞLEV	ESP32-S3 PINI	KAYNAK
Motor1 (SOL) sürücü IN_A / IN_B	`GPIO12` / `GPIO13`	`esp32s3_config.h`
Motor2 (SAĞ) sürücü IN_A / IN_B	`GPIO10` / `GPIO11`	`esp32s3_config.h`
Motor1 (SOL) encoder A / B	`GPIO6` / `GPIO7`	`esp32s3_config.h`
Motor2 (SAĞ) encoder A / B	`GPIO4` / `GPIO5`	`esp32s3_config.h`
Encoder Vcc (her ikisi)	`3V3` (GPIO değil, güç pini)	Board üzerindeki 3.3V çıkışı — 5V'tan asla beslenmez
I2C (gelecekteki IMU için, opsiyonel)	`SDA=GPIO8` / `SCL=GPIO9`	`esp32s3_config.h`

POLOLU 6 TELLI DEMET RENK KODU

Kırmızı=M+ · Siyah=M− · Sarı=Enc A · Beyaz=Enc B · Mavi=Enc Vcc(3.3V) · Yeşil=Enc GND

---

§05 Mekanik Montaj

Gövde, tek seferde değil **alttan yukarıya doğru katman katman** monte edilir; her katman bir öncekinin üzerine sarı spacer çubuklar ve somunlarla oturur.

1. **Katman 1 — Motorlar & sürücüler**

   İki Pololu motor ve iki BTS7960 sürücü en alt katmana sabitlenir. Motorların hangisinin SOL/SAĞ olacağı burada fiziksel olarak belirlenir — pin haritasıyla (§04) tutarlı olmalı.

2. **Katman 2 — ESP32-S3 & XL4015E1**

   Mikrodenetleyici ve güç regülatörü ikinci katmana yerleştirilir; motor/encoder kabloları bu katmana çıkar.

3. **Katman 3 — Raspberry Pi 4**

   Üçüncü katman RPi4'ü taşır; XL4015E1'in 5V çıkışı buraya USB-C ile bağlanır.

4. **Katman 4 — LIDAR**

   En üst katman LIDAR'ı taşır — 360° görüş açısı için diğer katmanların üzerinde, engelsiz bir konumda olmalı.

5. **Tekerlek montajı**

   Motor D-şaftına önce şaft adaptörü, sonra tekerlek geçirilir. **Bu arayüz, projenin en kritik tolerans noktasıdır** — aşağıdaki tasarım notuna dikkat edin.

TASARIM NOTU — ŞAFT ADAPTÖRÜ TOLERANSI

Motor şaftı 4mm D-profil, plastik (3D baskı) bir adaptöre *zorlanarak* geçirilirse, plastik mikro düzeyde deforme olup merkezden kayar — bu, tekerlek dönerken gözle görülür bir sallanmaya ("8 çizme") yol açar. Doğru yaklaşım: deliği şafttan hafif büyük (4.0–4.2mm) tasarlayıp hiç zorlamadan geçirmek, sabitlemeyi bir **set vidası + M3 somun cebi** (şaftın düz D-yüzeyine bastıran) ile yapmak, delik çevresinde en az 3–4mm dolu et kalınlığı bırakmak, ve baskı sonrası deliği gerçek bir 4mm matkap ucuyla raybalamak.

---

§06 ESP32-S3 Firmware Kurulumu

Yükleme yöntemi PlatformIO'dur (Arduino IDE değil) — micro-ROS yalnızca PlatformIO ile derlenebilir. ESP32-S3'ün dahili native USB'si sayesinde ayrı bir USB-TTL adaptörüne gerek yoktur.

1. **PlatformIO CLI kurulumu**

```
pip install platformio
```

2. **linorobot2_hardware deposunu klonla**

```
git clone -b humble https://github.com/linorobot/linorobot2_hardware.git
cd linorobot2_hardware/firmware
```

3. **Doğru config'i doğrula**

`config.h` içinde `USE_ESP32S3_CONFIG` tanımlı olmalı; motor/pin/kalibrasyon değerleri `config/custom/esp32s3_config.h` içindedir (§04'teki pin haritasıyla aynı).

4. **Derle ve yükle**

```
pio run -e esp32s3 -t upload --upload-port /dev/ttyACM0
```

Native USB portunda otomatik reset devresi olmadığından, yükleme sırasında **BOOT** tuşuna basılı tutup **RESET**'e dokunmak gerekebilir.

5. **Doğru portu ayırt et**

Kartın iki USB-C portu vardır: `idVendor=303a` (native USB — Serial/micro-ROS için kullanılan budur) ve `idVendor=1a86` (UART köprü — sadece flaşlama). Karıştırmayın.

---

§07 Raspberry Pi 4 Kurulumu

1

İşletim sistemi

Raspberry Pi Imager ile **Ubuntu Server 22.04.5 LTS (64-bit)** — Desktop değil, 32-bit değil. Gelişmiş ayarlardan SSH + kullanıcı + WiFi önceden yapılandırılabilir.

2

ROS 2 Humble

```
sudo apt install ros-humble-ros-base
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

3

micro-ROS agent

```
mkdir -p ~/microros_ws/src && cd ~/microros_ws/src
git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git
cd ~/microros_ws && colcon build
source install/local_setup.bash
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
```

Düşük RAM'li kartlarda (1-2GB) derleme sırasında geçici bir swap dosyası gerekebilir — `/etc/fstab`'a eklenmezse her reboot'ta kaybolur.

4

Tam linorobot2 kurulumu (Nav2/SLAM/URDF)

```
wget https://raw.githubusercontent.com/linorobot/linorobot2/humble/install_linorobot2.bash
bash install_linorobot2.bash 2wd a1
```

İkinci argüman LIDAR modelidir — desteklenenler arasında `a1`, `s1`, `s2`, `s3` ve diğerleri vardır (`sllidar_ros2` paketi üzerinden).

5

Bringup

```
ros2 launch linorobot2_bringup bringup.launch.py micro_ros_baudrate:=921600
```

`micro_ros_baudrate` mutlaka `esp32s3_config.h`'deki `BAUDRATE` (921600) ile eşleşmelidir — varsayılan (115200) farklıdır.

---

§08 Kalibrasyon

ROS2'ye hiç bağlanmadan, repo'nun kendi `calibration/` firmware'iyle motor/tekerlek değerleri ölçülür.

PARAMETRE	NASIL ÖLÇÜLÜR	BU PROJEDE ÖLÇÜLEN DEĞER
`MOTOR_MAX_RPM`	Üreticinin datasheet'i	`100 RPM (yüksüz, 12V)`
`COUNTS_PER_REV`	Seri monitörden `sample` komutu, motor başına	`M1=2103 · M2=2146`
`WHEEL_DIAMETER`	Kumpas/cetvel ile — **çap**, yarıçap değil	`0.066 m`
`LR_WHEELS_DISTANCE`	Sol-sağ tekerlek merkezleri arası	`0.160 m`
`K_P / K_I / K_D`	Stok değerlerle başla, sadece gerekirse ayarla	`0.6 / 0.8 / 0.5`

BILINEN FIRMWARE HATASI (UPSTREAM)

Kalibrasyon firmware'inin `sampleMotors()` fonksiyonu, `spin()` çağrısını bir `while(true)` döngüsü içinde saniyede yüz binlerce kez tekrarlarsa, ESP32'nin PWM donanımı (LEDC) geçerli bir sinyal üretemez ve motor **hiç dönmez**. Çözüm: `spin()`'i döngünün *dışına, öncesine* taşımak — döngü sadece zamanlamayı kontrol etmeli, PWM komutunu tekrar tekrar göndermemelidir.

§09 İlk Çalıştırma ve Doğrulama

Güç vermeden önce ve her aşamada, sırayla doğrulanması gereken kontrol listesi:

- [ ] Multimetre ile XL4015E1 çıkışını RPi4'e bağlamadan **önce** ölç — 5.0–5.2V aralığında olmalı
- [ ] Tüm GND'lerin (batarya, regülatör, her iki sürücü, ESP32-S3) tek noktada ortak olduğunu süreklilik testiyle doğrula
- [ ] Encoder Vcc'nin 3.3V'a bağlı olduğunu, **ESP32'nin RST pinine değil** , teyit et (bu iki pin çoğu kartta yan yanadır — kolay karışır)
- [ ] Motor sürücüsünü güçlendirmeden ESP32↔RPi4 micro-ROS testine **başlama** — güçsüz sürücü, seri hatta elektriksel gürültü bindirip agent'ın çökmesine yol açabilir
- [ ] `ros2 topic echo /odom/unfiltered` ile canlı encoder verisini doğrula
- [ ] Teleop ile dört yönü de (ileri/geri/sol/sağ) tek tek test et, motor kimliğinin (SOL=Motor1/SAĞ=Motor2) doğru olduğunu gözle onayla
- [ ] LIDAR'ı tek başına başlatıp (`ros2 launch sllidar_ros2 sllidar_a1_launch.py`) "health status: OK" logunu gör
- [ ] Tam `bringup.launch.py` + `slam.launch.py` ile RViz'de canlı harita oluşumunu izle

§10 Bilinen Sorunlar ve Dersler

Bu projenin geliştirilmesi sırasında karşılaşılan ve kök nedeni bulunan sorunlar — sıfırdan aynı yığını kuran biri için en değerli kısım muhtemelen burasıdır.

ELEKTRIKSEL

Ortak GND unutulursa

Belirli yön komutları çalışırken diğerleri "tekler", akım 0A okunur. Her fiziksel yeniden kablolamadan sonra ortak GND'yi elle kontrol edin.

FIRMWARE

Encoder pull-down varsayılanı

`ESP32Encoder` kütüphanesinin varsayılanı pull-DOWN'dır; açık-kolektör encoderlar bununla hiç veri üretmez. `useInternalWeakPullResistors = puType::up;` şart.

FIRMWARE

PWM tight-loop hatası

`spin()`'in bir döngü içinde tekrar tekrar çağrılması LEDC donanımını bozar. Bkz. §08.

KABLOLAMA

RST / 3.3V pin karışıklığı

Bu iki pin çoğu ESP32 kartında yan yanadır. Encoder Vcc'nin RST'ye bağlanması, kartın kendini sürekli resetlemesine ve gürültülü encoder verisine yol açar.

KINEMATIK

Motor1/Motor2 kimlik takası

Yalnızca düz gitmek doğru ama dönüşler ters çalışıyorsa, tek bir motorun `_INV` bayrağını değil, **Motor1↔Motor2'nin tüm config bloğunu** (pin, encoder, INV) birlikte takas edin.

PERFORMANS

RPi4 + RViz üzerinden WiFi

RViz'i WiFi üzerinden sürekli açık bırakmak, ROS2/DDS'in geri basınç (back-pressure) yaşamasına ve `micro_ros_agent`'ın senkron kaybetmesine yol açabilir. Sadece kontrol için aç, sonra kapat.

MEKANIK

Şaft-adaptör toleransı

Bkz. §05 — zorlanarak takılan plastik adaptör tekerlek sallanmasının en sık nedenidir.

GÜÇ

Sürücü güçsüzken test etmeyin

Sadece ESP32+RPi4 test edilirken BTS7960 güçsüz bırakılırsa, PWM hattı seri iletişime gürültü bindirip `NotEnoughMemoryException` ile agent'ı çökertebilir.

§11 Ekler — Tam Dosya İçerikleri

Bu kılavuzda referans verilen iki dosyanın **tam, değiştirilmemiş içeriği** — kopyalayıp doğrudan kullanabilirsiniz.

`esp32s3_config.h` — linorobot2_hardware/config/custom/

Bu projede fiilen kullanılan, doğrulanmış pin/kalibrasyon değerleriyle.

```
// ================================================================================
// LinoRobot2 Projesi — ESP32-S3 Donanım Yapılandırması
// ================================================================================
// Bu robotta fiilen kullanılan yapılandırma dosyasıdır. linorobot2_hardware
// projesinin ESP32-S3 config şablonu temel alınmış, aşağıdaki değerler bu
// robota özel olarak fiziksel ölçüm ve kalibrasyonla belirlenmiştir:
//
//   Motor            : 2x Pololu 25D, 99:1 dişli oranı, #3219 (48 CPR encoder)
//   Sürücü           : 2x BTS7960 (IBT_2)
//   MOTOR_MAX_RPM    : Pololu #3219 datasheet'inden (100 RPM, yüksüz @ 12V)
//   COUNTS_PER_REV1/2: `calibration/` firmware'i ile `sample` komutuyla ölçüldü
//                      (Motor1/SOL=2103, Motor2/SAĞ=2146)
//   WHEEL_DIAMETER   : Kumpasla fiziksel ölçüm (66mm = 0.066m)
//   Pin ataması      : Motor1(SOL)=GPIO12/13+encoder GPIO6/7,
//                      Motor2(SAĞ)=GPIO10/11+encoder GPIO4/5
//   WIFI_* / AGENT_IP/ SYSLOG_*: Bu projede kullanılmıyor — ESP32↔RPi4
//                      bağlantısı USB seri (micro-ROS, 921600 baud) üzerinden
//
// Aşağıdaki lisans bildirimi, temel alınan orijinal linorobot2_hardware
// şablonuna aittir (Apache 2.0 — değiştirilmeden korunmuştur).
// ================================================================================
//
// Copyright (c) 2021 Juan Miguel Jimeno
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

#ifndef ESP32S3_CONFIG_H
#define ESP32S3_CONFIG_H

#define LED_PIN LED_BUILTIN //used for debugging status

//uncomment the base you're building
#define LINO_BASE DIFFERENTIAL_DRIVE       // 2WD and Tracked robot w/ 2 motors
// #define LINO_BASE SKID_STEER            // 4WD robot
// #define LINO_BASE MECANUM               // Mecanum drive robot

//uncomment the motor driver you're using
// #define USE_GENERIC_2_IN_MOTOR_DRIVER      // Motor drivers with 2 Direction Pins(INA, INB) and 1 PWM(ENABLE) pin ie. L298, L293, VNH5019
// #define USE_GENERIC_1_IN_MOTOR_DRIVER   // Motor drivers with 1 Direction Pin(INA) and 1 PWM(ENABLE) pin.
#define USE_BTS7960_MOTOR_DRIVER        // BTS7970 Motor Driver using A4950 (<40V) module or DRV8833 (<10V)
// #define USE_ESC_MOTOR_DRIVER            // Motor ESC for brushless motors

//uncomment the IMU you're using
// #define USE_GY85_IMU
// #define USE_MPU6050_IMU
// #define USE_MPU9150_IMU
// #define USE_MPU9250_IMU
// #define USE_QMI8658_IMU
// #define USE_HMC5883L_MAG
// #define USE_AK8963_MAG
// #define USE_AK8975_MAG
// #define USE_AK09918_MAG
// #define USE_QMC5883L_MAG
// #define MAG_BIAS { 0, 0, 0 }
// #define IMU_TWEAK {}
// #define MAG_TWEAK {}

#define ACCEL_COV { 0.01, 0.01, 0.01 }
#define GYRO_COV { 0.001, 0.001, 0.001 }
#define ORI_COV { 0.01, 0.01, 0.01 }
#define MAG_COV { 1e-12, 1e-12, 1e-12 }
#define POSE_COV { 0.001, 0.001, 0.001, 0.001, 0.001, 0.001 }
#define TWIST_COV { 0.001, 0.001, 0.001, 0.003, 0.003, 0.003 }

#define K_P 0.6                            // P constant
```

```
#define K_I 0.8                           // I constant
#define K_D 0.5                           // D constant

/*
ROBOT ORIENTATION
         FRONT
    MOTOR1  MOTOR2  (2WD/ACKERMANN)
    MOTOR3  MOTOR4  (4WD/MECANUM)
         BACK
*/

//define your robot' specs here
#define MOTOR_MAX_RPM 100                 // motor's max RPM (Pololu #3207, 100 RPM no-load @ 12V)
#define MAX_RPM_RATIO 0.85                // max RPM allowed for each MAX_RPM_ALLOWED = MOTOR_MAX_RPM * MAX_RPM_RATIO
#define MOTOR_OPERATING_VOLTAGE 12        // motor's operating voltage (used to calculate max RPM)
#define MOTOR_POWER_MAX_VOLTAGE 12        // max voltage of the motor's power source (used to calculate max RPM)
#define MOTOR_POWER_MEASURED_VOLTAGE 12   // current voltage reading of the power connected to the motor (used for calibration)
#define COUNTS_PER_REV1 2103               // wheel1 encoder's no of ticks per rev (reverted 2026-07-23 — recalibrated value 2186 did not fix left-lag imbalance, reverted back)
#define COUNTS_PER_REV2 2146               // wheel2 encoder's no of ticks per rev (reverted 2026-07-23, same reason)
#define COUNTS_PER_REV3 450               // wheel3 encoder's no of ticks per rev
#define COUNTS_PER_REV4 450               // wheel4 encoder's no of ticks per rev
#define WHEEL_DIAMETER 0.066              // wheel's diameter in meters
#define LR_WHEELS_DISTANCE 0.160          // distance between left and right wheels
#define PWM_BITS 10                       // PWM Resolution of the microcontroller
#define PWM_FREQUENCY 20000               // PWM Frequency

// INVERT ENCODER COUNTS
#define MOTOR1_ENCODER_INV true
#define MOTOR2_ENCODER_INV false
#define MOTOR3_ENCODER_INV false
#define MOTOR4_ENCODER_INV false

// INVERT MOTOR DIRECTIONS
#define MOTOR1_INV true
#define MOTOR2_INV false
#define MOTOR3_INV false
#define MOTOR4_INV false

// ENCODER PINS
#define MOTOR1_ENCODER_A 6
#define MOTOR1_ENCODER_B 7

#define MOTOR2_ENCODER_A 4
#define MOTOR2_ENCODER_B 5

#define MOTOR3_ENCODER_A 39
#define MOTOR3_ENCODER_B 40

#define MOTOR4_ENCODER_A 41
#define MOTOR4_ENCODER_B 42

// MOTOR PINS
#ifdef USE_GENERIC_2_IN_MOTOR_DRIVER
  #define MOTOR1_PWM 21
  #define MOTOR1_IN_A 20
  #define MOTOR1_IN_B 1

  #define MOTOR2_PWM 5
  #define MOTOR2_IN_A 6
  #define MOTOR2_IN_B 8

  #define MOTOR3_PWM 22
  #define MOTOR3_IN_A 23
  #define MOTOR3_IN_B 0

  #define MOTOR4_PWM 4
  #define MOTOR4_IN_A 3
  #define MOTOR4_IN_B 2

  #define PWM_MAX (pow(2, PWM_BITS) - 1)
  #define PWM_MIN -PWM_MAX
#endif

#ifdef USE_GENERIC_1_IN_MOTOR_DRIVER
  #define MOTOR1_PWM 21
  #define MOTOR1_IN_A 20
  #define MOTOR1_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR2_PWM 5
  #define MOTOR2_IN_A 6
  #define MOTOR2_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR3_PWM 22
  #define MOTOR3_IN_A 23
  #define MOTOR3_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR4_PWM 4
  #define MOTOR4_IN_A 3
  #define MOTOR4_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define PWM_MAX (pow(2, PWM_BITS) - 1)
  #define PWM_MIN -PWM_MAX
#endif
```

```
#ifdef USE_BTS7960_MOTOR_DRIVER
  #define MOTOR1_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR1_IN_A 12
  #define MOTOR1_IN_B 13

  #define MOTOR2_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR2_IN_A 10
  #define MOTOR2_IN_B 11

  #define MOTOR3_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR3_IN_A 14
  #define MOTOR3_IN_B 15

  #define MOTOR4_PWM -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR4_IN_A 16
  #define MOTOR4_IN_B 17

  #define PWM_MAX (pow(2, PWM_BITS) - 1)
  #define PWM_MIN -PWM_MAX
#endif

#ifdef USE_ESC_MOTOR_DRIVER
  #define MOTOR1_PWM 21
  #define MOTOR1_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR1_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR2_PWM 5
  #define MOTOR2_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR2_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR3_PWM 22
  #define MOTOR3_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR3_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define MOTOR4_PWM 4
  #define MOTOR4_IN_A -1 //DON'T TOUCH THIS! This is just a placeholder
  #define MOTOR4_IN_B -1 //DON'T TOUCH THIS! This is just a placeholder

  #define PWM_MAX 400
  #define PWM_MIN -PWM_MAX
#endif

#define AGENT_IP { 192, 168, 1, 100 }  // eg IP of the desktop computer
#define AGENT_PORT 8888
#define WIFI_SSID "WIFI_SSID"
#define WIFI_PASSWORD "WIFI_PASSWORD"
// Enable WiFi with null terminated list of multiple APs SSID and password
// #define WIFI_AP_LIST {{"WIFI_SSID", "WIFI_PASSWORD"}, {NULL}}
#define WIFI_MONITOR 2 // min. period to send wifi signal strength to syslog
// #define USE_ARDUINO_OTA
// #define USE_SYSLOG
#define SYSLOG_SERVER { 192, 168, 1, 100 }  // eg IP of the desktop computer
#define SYSLOG_PORT 514
#define DEVICE_HOSTNAME "esp32s3"
#define APP_NAME "hardware"
// #define USE_LIDAR_UDP
#define LIDAR_RXD 2
// #define LIDAR_PWM 15
#define LIDAR_SERIAL 1 // uart number
#define LIDAR_BAUDRATE 230400
#define LIDAR_SERVER { 192, 168, 1, 100 }  // eg IP of the desktop computer
#define LIDAR_PORT 8889
#define BAUDRATE 921600
#define SDA_PIN 8 // specify I2C pins
#define SCL_PIN 9
#define NODE_NAME "esp32s3"
// #define TOPIC_PREFIX "esp32s3/"

// battery voltage ADC pin
#define BATTERY_PIN 1
// 3.3V ref, 12 bits ADC, 33k + 10k voltage divider
// #define USE_ADC_LUT
#ifdef USE_ADC_LUT
const int16_t ADC_LUT[4096] = { /* insert adc_calibrate data here */ };
#define BATTERY_ADJUST(v) (ADC_LUT[v] * (3.3 / 4096 * (33 + 10) / 10 * 1.0))
#else
#define BATTERY_ADJUST(v) ((v) * (3.3 / 4096 * (33 + 10) / 10))
#endif
// #define USE_INA219
// #define TRIG_PIN 31 // ultrasonic sensor HC-SR04
// #define ECHO_PIN 32
#define USE_SHORT_BRAKE // for shorter stopping distance
// #define WDT_TIMEOUT 60 // Sec
#define BOARD_INIT { \
    Wire.begin(SDA_PIN, SCL_PIN); \
    Wire.setClock(400000); \
}
// #define BOARD_INIT_LATE {}
// #define BOARD_LOOP {}

#ifdef USE_SYSLOG
#define RCCHECK(fn) { rcl_ret_t temp_rc = fn; if((temp_rc != RCL_RET_OK)){ \
```

```
    syslog(LOG_ERR, "%s RCCHECK failed %d", __FUNCTION__, temp_rc); \
    }}
#else
#define RCCHECK(fn) { rcl_ret_t temp_rc = fn; if((temp_rc != RCL_RET_OK)){ \
    flashLED(3); \
    }} // do not block
#endif

#endif
```

install_linorobot2.bash — RPi4 kurulum betiği, proje notuyla

Betik mantığı upstream'den değiştirilmemiştir (güvenilirlik için); başına bu robotta tam olarak nasıl çalıştırıldığını açıklayan bir proje notu eklenmiştir.

```
#!/usr/bin/env bash
# ================================================================================
# LinoRobot2 Projesi — Raspberry Pi 4 Kurulum Notu
# ================================================================================
# Bu robotta şu şekilde çalıştırılmıştır:
#
#     $ bash install_linorobot2.bash 2wd a1
#
# (2wd = diferansiyel sürüşlü taban; a1 = RPLIDAR A1M8. Proje başlangıçta
#  RPLIDAR S1 ile kurulmuş, sonradan A1M8'e geçilmiş — "s1" argümanıyla da
#  aynı script sorunsuz çalışmaktadır, ikisi de sllidar_ros2 sürücüsünü kurar.)
#
# Betiğin kendisi resmi linorobot2 deposundan (humble dalı) değiştirilmeden
# alınmıştır — aşağıdaki lisans/telif bildirimi orijinal yazara aittir.
# ================================================================================
#
# Copyright (c) 2021 Juan Miguel Jimeno
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http:#www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

set -e

ROSDISTRO="$(printenv ROS_DISTRO)"
BASE=$1
LASER_SENSOR=$2
DEPTH_SENSOR=$3
ARCH="$(uname -m)"
WORKSPACE="$HOME/linorobot2_ws"

ROBOT_TYPE_ARRAY=(2wd 4wd mecanum)
DEPTH_SENSOR_ARRAY=(realsense zed zedm zed2 zed2i oakd oakdlite oakdpro)
LASER_SENSOR_ARRAY=(ydlidar xv11 ld06 ld19 stl27l a1 a2 a3 c1 s1 s2 s3)
LASER_SENSOR_ARRAY+=(${DEPTH_SENSOR_ARRAY[@]})

if [ -z "$LASER_SENSOR" ]
    then
        LASER_SENSOR=""
fi

if [ -z "$DEPTH_SENSOR" ]
    then
        DEPTH_SENSOR=""
fi

function install_cuda_jetson {
    wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2004/sbsa/cuda-ubuntu2004.pin
    sudo mv cuda-ubuntu2004.pin /etc/apt/preferences.d/cuda-repository-pin-600
    wget http://developer.download.nvidia.com/compute/cuda/11.4.2/local_installers/cuda-repo-ubuntu2004-11-4-local_11.4.2-470.57.02-1_arm64.deb
    sudo dpkg -i cuda-repo-ubuntu2004-11-4-local_11.4.2-470.57.02-1_arm64.deb
    sudo apt-key add /var/cuda-repo-ubuntu2004-11-4-local/7fa2af80.pub #verify this
    sudo apt-get update
    sudo apt-get -y install cuda
    # Errors were encountered while processing:
    #  /tmp/apt-dpkg-install-TvUCLd/14-libnvidia-compute-470_470.57.02-0ubuntu1_arm64.deb
    #  /tmp/apt-dpkg-install-TvUCLd/18-libnvidia-gl-470_470.57.02-0ubuntu1_arm64.deb
    # E: Sub-process /usr/bin/dpkg returned an error code (1)
}

function install_xv11 {
    cd $WORKSPACE
    git clone https://github.com/mjstn/xv_11_driver src/xv_11_driver
    colcon build
```

```
    source $WORKSPACE/install/setup.bash
}

function install_ydlidar {
    cd /tmp
    git clone https://github.com/YDLIDAR/YDLidar-SDK.git
    mkdir YDLidar-SDK/build
    cd YDLidar-SDK/build
    cmake ..
    make
    sudo make install
    cd $WORKSPACE
    git clone https://github.com/YDLIDAR/ydlidar_ros2_driver src/ydlidar_ros2_driver
    chmod 0777 src/ydlidar_ros2_driver/startup/*
    sudo echo  'KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}=="ea60", MODE:="0666", GROUP:="dialout",  SYMLINK+="ydlidar"' >/etc/udev/rules.d/ydlidar.rules
    sudo echo  'KERNEL=="ttyACM*", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="5740", MODE:="0666", GROUP:="dialout",  SYMLINK+="ydlidar"' >/etc/udev/rules.d/ydlidar-V2.rules
    sudo echo  'KERNEL=="ttyUSB*", ATTRS{idVendor}=="067b", ATTRS{idProduct}=="2303", MODE:="0666", GROUP:="dialout",  SYMLINK+="ydlidar"' >/etc/udev/rules.d/ydlidar-2303.rules
    colcon build --symlink-install
    source $WORKSPACE/install/setup.bash
}

function install_ldlidar_stl_ros2 {
    cd $WORKSPACE
    git clone https://github.com/hippo5329/ldlidar_stl_ros2.git src/ldlidar_stl_ros2
    colcon build
    source $WORKSPACE/install/setup.bash
    cd /tmp
    wget https://raw.githubusercontent.com/linorobot/ldlidar/ros2/ldlidar.rules
    sudo cp ldlidar.rules /etc/udev/rules.d
}

function install_ld06 {
    install_ldlidar_stl_ros2
}

function install_ld19 {
    install_ldlidar_stl_ros2
}

function install_stl27l {
    install_ldlidar_stl_ros2
}

function install_sllidar_ros2 {
    cd $WORKSPACE
    git clone https://github.com/Slamtec/sllidar_ros2.git
    colcon build
    source $WORKSPACE/install/setup.bash
    sudo cp sllidar_ros2/scripts/rplidar.rules /etc/udev/rules.d
}

function install_a1 {
    install_sllidar_ros2
}

function install_a2 {
    install_sllidar_ros2
}

function install_a3 {
    install_sllidar_ros2
}

function install_c1 {
    install_sllidar_ros2
}

function install_s1 {
    install_sllidar_ros2
}

function install_s2 {
    install_sllidar_ros2
}

function install_s3 {
    install_sllidar_ros2
}

function install_realsense {
    sudo apt install -y ros-$ROS_DISTRO-realsense2-camera
    cd /tmp
    wget https://raw.githubusercontent.com/IntelRealSense/librealsense/master/config/99-realsense-libusb.rules
    sudo cp 99-realsense-libusb.rules /etc/udev/rules.d
}

function install_astra {
    cd $WORKSPACE
    sudo apt install -y libuvc-dev libopenni2-dev
    git clone https://github.com/linorobot/ros_astra_camera src/ros_astra_camera
    sudo cp src/ros_astra_camera/56-orbbec-usb.rules /etc/udev/rules.d/
    colcon build
    source $WORKSPACE/install/setup.bash
```

```
}

function install_zed {
    cd /tmp
    if [[ -f /etc/nv_tegra_release ]]
        then
            #TODO ADD CUDA INSTALLATION HERE
            wget https://download.stereolabs.com/zedsdk/3.5/jp45/jetsons -O zed_sdk
    elif lspci | grep VGA | grep -o NVIDIA
        then
            wget https://download.stereolabs.com/zedsdk/3.5/cu111/ubuntu20 -O zed_sdk
    else
        echo "Linux Machine not supported by Zed Camera"
        exit 1
    fi

    chmod +x zed_sdk
    ./zed_sdk -- silent
    cd $WORKSPACE

    git clone https://github.com/stereolabs/zed-ros2-wrapper src/zed-ros2-wrapper
    git clone https://github.com/ros-perception/image_common -b $ROS_DISTRO src/image_common #https://github.com/stereolabs/zed-ros2-wrapper#image-transport-and-topic-subscriptions
    rosdep install --from-paths src --ignore-src -r -y
    colcon build --symlink-install --cmake-args=-DCMAKE_BUILD_TYPE=Release
    # colcon build --symlink-install --cmake-args=-DCMAKE_BUILD_TYPE=Release --cmake-args=-DCUDA_TOOLKIT_ROOT_DIR=/usr/local/cuda-11.4
    source $WORKSPACE/install/setup.bash
    source ~/.bashrc
}

function install_zedm {
    install_zed
}

function install_zed2 {
    install_zed
}

function install_zed2i {
    install_zed
}

function install_oakd {
    echo 'SUBSYSTEM=="usb", ATTRS{idVendor}=="03e7", MODE="0666"' | sudo tee /etc/udev/rules.d/80-movidius.rules
    sudo udevadm control --reload-rules && sudo udevadm trigger
    sudo apt install ros-$ROS_DISTRO-depthai-ros
}

function install_oakdlite {
    install_oakd
}

function install_oakdpro {
    install_oakd
}

if [[ "$ROSDISTRO" == "" || "$ROSDISTRO" == "<unknown>" ]]
    then
        echo "No ROS2 distro detected"
        echo "Try running $ source /opt/ros/<ros_distro>/setup.bash and try again."
        exit 1
fi

if [ "$*" == "" ]
    then
        echo "No arguments provided"
        echo
        echo "Example: $ bash install_linorobot2.bash 2wd a1"
        echo "Example: $ bash install_linorobot2.bash 2wd a1 realsense"
        echo "Example: $ bash install_linorobot2.bash 2wd - realsense"
        echo "Example: $ bash install_linorobot2.bash 2wd"

        echo
        exit 1
fi

if [[ "$BASE" != "ci" ]] && !(printf '%s\n' "${ROBOT_TYPE_ARRAY[@]}" | grep -xq $BASE)
    then
        echo "Invalid linorobot base: $1"
        echo
        echo "Valid Options:"
        for key in "${!ROBOT_TYPE_ARRAY[@]}"; do echo "${ROBOT_TYPE_ARRAY[$key]}"; done
        echo
        exit 1
fi

if [[ "$BASE" != "ci" && "$LASER_SENSOR" != "" && "$LASER_SENSOR" != "-" ]] && !(printf '%s\n' "${LASER_SENSOR_ARRAY[@]}" | grep -xq $LASER_SENSOR)
    then
        echo "Invalid linorobot2 laser sensor: $LASER_SENSOR"
        echo
        echo "Valid Options:"
        for key in "${!LASER_SENSOR_ARRAY[@]}"; do echo "${LASER_SENSOR_ARRAY[$key]}"; done
        echo
        exit 1
```

```
fi

if [[ "$BASE" != "ci" && "$DEPTH_SENSOR" != "" ]] && !(printf '%s\n' "${DEPTH_SENSOR_ARRAY[@]}" | grep -xq $DEPTH_SENSOR)
    then
        echo "Invalid linorobot2 depth sensor: $DEPTH_SENSOR"
        echo
        echo "Valid Options:"
        for key in "${!DEPTH_SENSOR_ARRAY[@]}"; do echo "${DEPTH_SENSOR_ARRAY[$key]}"; done
        echo
        exit 1
fi

if [[ "$BASE" != "ci" ]]
    then
        echo
        echo "You are installing linorobot2 on your robot computer."
        echo
        echo "===========SUMMARY==========="
        echo "ROBOT TYPE   : $BASE"
        echo "LASER SENSOR : $LASER_SENSOR"
        echo "DEPTH SENSOR : $DEPTH_SENSOR"
        echo
        echo "This installer will edit your ~/.bashrc."
        echo "Create a linorobot2_ws on your $HOME directory."
        echo "Install linorobot2 ROS2 dependencies."
        echo "Install udev rules on /etc/udev/rules.d folder."
        echo -n "Enter [y] to continue. "
        read reply
        if [[ "$reply" != "y" && "$reply" != "Y" ]]
            then
                echo "Exiting now."
                exit 1
        fi
fi

echo
echo "INSTALLING NOW...."
echo

#### 1.1 Source your ROS2 distro and workspace
cd $HOME
mkdir -p $WORKSPACE/src
source /opt/ros/$ROS_DISTRO/setup.bash
cd $WORKSPACE
colcon build
source $WORKSPACE/install/setup.bash

#### 1.2/1.3 Install LIDAR/Depth Sensor ROS2 drivers:
if (printf '%s\n' "${LASER_SENSOR_ARRAY[@]}" | grep -xq $LASER_SENSOR)
    then
        install_$LASER_SENSOR
fi

if (printf '%s\n' "${DEPTH_SENSOR_ARRAY[@]}" | grep -xq $DEPTH_SENSOR)
    then
        install_$DEPTH_SENSOR
fi

if [[ "$BASE" == "ci" ]]
    then
        for key in "${!LASER_SENSOR_ARRAY[@]}"; do  install_${LASER_SENSOR_ARRAY[$key]}; done
fi

#### 1.4 Download and install micro-ROS:
cd $WORKSPACE
git clone -b $ROS_DISTRO https://github.com/micro-ROS/micro_ros_setup.git src/micro_ros_setup
sudo apt install -y python3-vcstool build-essential
sudo apt update && rosdep update
rosdep install --from-path src --ignore-src -y
colcon build
source $WORKSPACE/install/setup.bash

#### 1.5 Setup micro-ROS agent:
ros2 run micro_ros_setup create_agent_ws.sh
ros2 run micro_ros_setup build_agent.sh
source $WORKSPACE/install/setup.bash

#### 2.1 Download linorobot2:
cd $WORKSPACE
git clone -b $ROS_DISTRO https://github.com/linorobot/linorobot2 src/linorobot2

#### 2.2 Ignore Gazebo Packages on robot computer (optional)
cd $WORKSPACE/src/linorobot2/linorobot2_gazebo
touch COLCON_IGNORE

#### 2.3 Install linorobot2 package:
cd $WORKSPACE
rosdep update && rosdep install --from-path src --ignore-src -y --skip-keys microxrcedds_agent
colcon build
source $WORKSPACE/install/setup.bash

## ENV Variables
if [[ "$BASE" != "ci" ]]
```

```
    then
        ### 1. Robot Type
        echo "export LINOROBOT2_BASE=$BASE" >> ~/.bashrc
        ### 2. Sensors
        if [[ "$LASER_SENSOR" != "-" ||  "$LASER_SENSOR" != "" ]]
            then
                echo "export LINOROBOT2_LASER_SENSOR=$LASER_SENSOR" >> ~/.bashrc
        fi

        if [[ "$DEPTH_SENSOR" != "-" ||  "$DEPTH_SENSOR" != "" ]]
            then
            echo "export LINOROBOT2_DEPTH_SENSOR=$DEPTH_SENSOR" >> ~/.bashrc
        fi
        echo
        echo "Do you want to add sourcing of linorobot2_ws on your ~/.bashrc?"
        echo -n "Yes [y] or No [n]: "
        read reply
        if [[ "$reply" == "y" || "$reply" == "Y" ]]
            then
                echo "source \$HOME/linorobot2_ws/install/setup.bash" >> ~/.bashrc
        else
            echo
            echo "Remember to run $ source ~/linorobot2_ws/install/setup.bash every time you open a terminal."
        fi
fi

echo
echo "INSTALLATION DONE."
echo
echo "Restart your robot computer now."
```